TECHNISCHE HOCHSCHULE MITTELHESSEN

CAMPUS GIESSEN

MNI
Mathematik, Naturwissenschaften und Informatik

Master Thesis

Integration of model-driven development approaches into the domain of mobile applications

Classification of an interpreter implementation for mobile applications

Analysis of a model interpreter on the basis of an existing implementation

zur Erlangung des akademischen Grades

Master of Science

eingereicht im Fachbereich Mathematik, Naturwissenschaften und Informatik an der Technische Hochschule Mittelhessen

von

Samuel Schepp

March 4, 2019

Referent: Prof. Dr. Peter Kneisel

Korreferent: Dennis Priefer

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Gießen, den March 4, 2019 Samuel Schepp

This thesis describes configuration-based mobile applications and their infrastructure. In addition, a possible way of implementing a configuration system is discussed. Finally, the implementation is compared to the approach of established literature by classifying the model and interpreter.

Furthermore, there is an evaluation of the use case of interpreting and generated apps.

Contents

1 Introduction 1
- 1.1 Problem definition . . . 1
- 1.2 Motivation . . . 2
- 1.3 Distinction . . . 3
- 1.4 Structure . . . 3

2 Background 5
- 2.1 Fabrik19 . . . 5
  - 2.1.1 Markets . . . 5
  - 2.1.2 Customers . . . 6
- 2.2 Mobilitysuite . . . 6
  - 2.2.1 Use cases . . . 6
  - 2.2.2 Interoperability . . . 6

3 Concept 7
- 3.1 Models . . . 7
  - 3.1.1 General models specification . . . 7
  - 3.1.2 Descriptive Models . . . 7
  - 3.1.3 Prescriptive Models . . . 7
- 3.2 Model driven development . . . 7
  - 3.2.1 Code generators . . . 7
  - 3.2.2 Code interpreters . . . 7
- 3.3 Implementation goals . . . 7
  - 3.3.1 Trade offs . . . 7

4 Implementation 9
- 4.1 Language . . . 9
- 4.2 Components . . . 9
  - 4.2.1 Nodes . . . 9
  - 4.2.2 Layouts . . . 9
  - 4.2.3 Database . . . 9
  - 4.2.4 Actions . . . 9
  - 4.2.5 Data proxy . . . 9

4.3 Infrastructure 9
4.3.1 Control center 9
4.3.2 Resource server 9
4.3.3 Image cruncher 9

5 Evaluation 11
5.1 Analysis 11
5.1.1 Benefits 11
5.1.2 Disadvantages 11
5.2 Classification 11
5.2.1 Approach 11
5.2.2 Model 11
5.3 Conclusion 11

Bibliography 13

Index of abbreviations 15

List of Figures 17

List of Tables 19

Listings 21

1 Introduction

Mobile application and digital media usage is still on the rising[Per]. Mobile applications are one of many possible ways to interact with customers and consumers. A web appearance on social media or a dedicated web page can also fulfill these needs. The way, the user interacts with a web page is more shallow, because of lower expectations. A user needs to log in most of the time.

A mobile application can enable user specific features without logging in, because the user can be identified with an identification token, that is generated, while the user installs the app. It is possible to activate special features like geofencing, push notifications or local data storage, that is synchronized with the platform cloud services (e.g. Apple iCloud or Google Drive).

A mobile app is integrated more deeply into the device, than a web app. However, web browser already support a lot of the features by now, that originally were only available to native apps. The current development indicates, that the differences will be further eliminated by the browser vendors.

1.1 Problem definition

Mobile applications have significant disadvantages, that push the development of web apps forward. Changes have to be approved by the phone vendors, in order to be able to release updates to the app. This causes development delays and reduces the ability to react to events or customer requirement changes.

All apps have to be kept in good condition, which creates more work, the more apps there need to be maintained. New operating system features have to be implemented in all of the apps, which can cause inconsistencies. All apps have to be tested in detail, to make sure that all apps implement the required change correctly. Platform changes can only be dealt with one by one. Developers need deep system knowledge to develop the apps. Every change has to go through one of each platform experts, so development is expensive.

An example for a change, that all apps should implement is the display notch. It causes the screen to extend to the edge of the display. However some of the screen is cut out to make room for sensors and the camera.

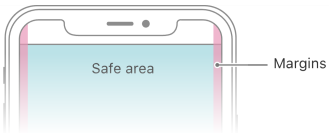

Figure 1.1: Display notch with safe area, Apple iPhone X

The notch is integrated to the default interface framework, so that developers can react accordingly and use the provided margins. This change requires all apps to implement a correct way to handle that. Developers have to go though every screen and correct the view.

1.2 Motivation

To be able to deliver the full mobile application experience, the app has to be native. This will change, when all of the phone's hardware interface is exposed to the web browser and user interface performance is better. But for now, the only way to use these features and be good on battery life and data usage is by providing a native app, that is written in the native language and uses features, directly provided by the phone vendors.

Web app wrapper like "ionic" are able to use these features and provide their own interface to the web app. In this way, a web app can in fact use restricted components like push notifications and beacons.

With this approach, every new app still has to be developed and old ones have to be maintained individually. In addition, it involves the dependence on third party framework vendors. Building apps with ionic creates a massive list of dependencies: ionic, angular, npm and nodejs. Native apps have the ability to reduce reliance, by only using native libraries, which are very comprehensive by default.

To be able to create many apps with little effort, a model driven approach can be considered. A well abstracted model can have the ability to be easy to understand by users and the platforms can bring their respected implementation in, so any app looks and feels native. A service can deliver pre-rendered sub-models, so that complex client workload can be moved to the server.

Existing apps are easy to maintain, because there is only one code base per platform. One bug fix applies to all apps. Also, it is easy for anyone to change small things like a

label, a background color or the sorting of a list. The goal is, to be able to configure new apps in one day, together with the customer.

1.3 Distinction

This thesis will not describe the development experience nor the initial expense of the project. Neither will this thesis evaluate financial aspects and at which point, the project was or is worth the development costs. The domain examination will be narrowed down to the two main ones, mobile phones and digital signage.

The thesis aims to answer the question:

> How can a code interpreter be integrated in the domain of mobile applications?

1.4 Structure

The thesis will give some information on the background of the interpreter. The company Fabrik19 GmbH, which is actively developing and maintaining an implementation, will be described and characterized. After that, the general concept of model driven development, code generators and interpreters are examined and presented.

The main content will then be about the concrete implementation, by describing the most significant design choices, overall infrastructure and usage examples. The focus will be on the two main platforms 'iOS' and 'Android'. The web and '.Net' clients are not considered, because they are not targeted for mobile devices and only support a subset of the model.

In the final chapter, the concept is applied to the implementation, to see how it fits the theoretical model of interpreters. Differences are worked out and discussed.

2 Background

The system, that is discussed here, is a model interpreter that is designed to run mobile applications on iOS and Android. The system, which includes the interpreters and the backends are called mos[1]. It is in active development and poweres over 50 apps that all are available in the Apple AppStore and Google PlayStore. The app categories range from retail to event to city marketing apps[2].

2.1 Fabrik19

The creator of the system is Fabrik 19 GmbH. The company is based in Gießen, Hesse, Germany and was founded 20XX by Mark Pralle and Sven Herchenhein.

2.1.1 Markets

Fabrik19 does not deliver apps directly to customers, but is partner of retail corporations and city marketing agencies. The current flagship is the Gießen App, which was developed in cooparation with DISTAMA GmbH. The App has 34,000 downloads and 20 active users per day.

Besides mobile apps, digital signage is another important market. The companion to the Gießen App is a stele that contains a large touchscreen and runs an app that is optimized for big touchscreens. It is located on the main pedestrian shopping street in Gießen and provides respective informations on the stores and points of interests[3]. Other stele systems are also active in Globus hardware stores and Globus grocery stores.

1 Mobility Suite
2 `https://fabrik19.de/technologie/`
3 `https://distama.de/bid-seltersweg/`

2.1.2 Customers

2.2 Mobility Suite

2.2.1 Use cases

2.2.2 Interoperability

lorem

3 Concept

3.1 Models

3.1.1 General models specification

3.1.2 Descriptive Models

3.1.3 Prescriptive Models

3.2 Model driven development

3.2.1 Code generators

3.2.2 Code interpreters

3.3 Implementation goals

Same day delivery

easy configuration

simple datasources

common app tasks simplified

content driven (wordpress, social media)

reactive form elements

3.3.1 Trade offs

boundary conditions

limits to development of app contents (app is out, but the customer needs more functionality, which is not possible: incomprehensible reasons for limits.)

4 Implementation

lorem ipsum

4.1 Language

4.2 Components

4.2.1 Nodes

4.2.2 Layouts

4.2.3 Database

4.2.4 Actions

4.2.5 Data proxy

4.3 Infrastructure

4.3.1 Control center

4.3.2 Resource server

4.3.3 Image cruncher

5 Evaluation

lorem ispum

5.1 Analysis

lorem ispum

5.1.1 Benefits

lorem ispum

5.1.2 Disadvantages

lorem ispum

5.2 Classification

5.2.1 Approach

5.2.2 Model

5.3 Conclusion

lorem ispum

Bibliography

[Per] Perkins, Kleiner: Internet Trends Report 2018, URL https://www.kleinerperkins.com/perspectives/internet-trends-report-2018/

Index of abbreviations

mos Mobility Suite

List of Figures

1.1 Display notch with safe area, Apple iPhone X 2

List of Tables

Listings

.